Programa:	Registry Medic 2.50	
Protección:	SVKP 1.3x + Trial 30 días + Opciones deshabilitadas	
Objetivos:	Desempakar y evitar la limitación	
URL descarga:	www.fu2000.com (creo)	
Dificultad:	No fue fácil…	
Herramientas:	OllyDbg + PEiD + Topo + Import Reconstructor.	
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos]	Fecha: 28-08-2005

Hola amigos..!!

Aquí estamos en la segunda parte del tut, no es normal que lo haga así pero visto que se podía alargar mucho el tema decidí partirlo en dos partes, nos habíamos quedado con el programa ya desempakado y la ***IAT*** reparada, ahora nos centraremos en la protección ***Trial***, la verdad es que me hizo dar muchas vueltas el tema ya que lo conseguía regisrtar pero no rulaba.

Crackeando Registry Medic 2.50

Comenzaremos carganso el programa en ***Ollydbg*** (recordad que trabajamos con el desempaquetado), estamos en el ***OEP*** y lo dejamos correr con ***F9***, no tarda mucho en aparecer el mensaje de Expired que ya vimos nos mostraba, al aceptarlo abre el cuadro de registro donde nos pide un nombre y un serial. Vale suficiente info, reiniciemos con ***Ctrl+F2*** antes de dar ***RUN*** pongamos un ***BP*** en ***MesssageBoxA*** para pillar el momento en el que nos va a mostrar el mensaje y sobretodo el lugar donde esto ocurra, puesto el ***BP*** damos ***RUN*** ahora sí:

```
0012FF60  004DE649  CALL to MessageBoxA from tut_inje.004DE644
0012FF64  001907A8  hOwner = 001907A8 ('Registry Medic',class='TApplication')
0012FF68  004DE75C  Text = "Trial Expired,please registration!"
0012FF6C  004DE754  Title = "Expired"
0012FF70  00000030  Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
```

Pues paró y como vemos podemos saber rápidamente con un simple vistazo al stack que la llamada se realizó en ***:4DE644***, pondremos un ***BP*** en la instrucción siguiente que es ***:4DE649***, vemos la zona:

```
Address   Hex dump    Disassembly                  Comment
004DE62A  .  64:8920  MOV DWORD PTR FS:[EAX],ESP
004DE62D  .  6A 30    PUSH 30                      Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
004DE62F  .  68 54E74D00 PUSH tut_inje.004DE754    Title = "Expired"
004DE634  .  68 5CE74D00 PUSH tut_inje.004DE75C    Text = "Trial Expired,please registration!"
004DE639  .  A1 D4794F00 MOV EAX,DWORD PTR DS:[4F79D4]
004DE63E  .  8B00     MOV EAX,DWORD PTR DS:[EAX]
004DE640  .  8B40 24  MOV EAX,DWORD PTR DS:[EAX+24]
004DE643  .  50       PUSH EAX                     hOwner
004DE644  .  E8 4F8DF2FF CALL <JMP.&user32.MessageBoxA> MessageBoxA
004DE649  .  8B0D D4794F0 MOV ECX,DWORD PTR DS:[4F79D4]  tut_inje.004F87D8
004DE64F  .  8B09     MOV ECX,DWORD PTR DS:[ECX]
```

Sin lugar a dudas es el lugar de la llamada, ponemos el ***BP*** como en la imagen y damos ***RUN***, aceptamos el mensaje y caemos en el ***BP***, es de resaltar que estamos en una rutina que empieza en ***:4DE608***, esto quizá nos sirva luego para algo de momento lo anotamos, dos calls después aparece el cuadro de registro, concretamente en ***:4DE663***, bien lo anotamos también, es evidente que estamos en la rutina de error, de momento no ponemos ningún serial ni nada, simplemente cancelamos y salimos con ***EAX==2*** interesante también lo anotamos por su puesto y s eguimos con el traceo, el primer ***RET*** nos pone un poco más abajo del ***OEP***, o se que uno de los primeros call se encarga de la limitación temporal, en concreto:

```
Address   Hex dump   Disassembly                  Comment
004DEBD9  8B00       MOV EAX,DWORD PTR DS:[EAX]
004DEBDB  BA 04EC4D00 MOV EDX,tut_inje.004DEC04   ASCII "Registry Medic"
004DEBE0  E8 E7F8F6FF CALL tut_inje.0044E4CC
004DEBE5  E8 96FBFFFF CALL tut_inje.004DE780      Controla el Trial
004DEBEA  A1 D4794F00 MOV EAX,DWORD PTR DS:[4F79D4]
004DEBEF  8B00       MOV EAX,DWORD PTR DS:[EAX]
```

Este es el culpable de ambos mensajes (el de expired y el cuadro de registro) lo que indica que los chequeos del timpo están en su interior, de momento le ponemos un ***BP*** como en la imagen y reiniciamos con ***Ctrl+F2***, damos ***RUN*** y paramos en él nuevamente, entremos:

```
Address   Hex dump   Disassembly                  Comment
004DE780  A1 D4774F00 MOV EAX,DWORD PTR DS:[4F77D4]
004DE785  8038 00    CMP BYTE PTR DS:[EAX],0
004DE788  75 10      JNZ SHORT tut_inje.004DE79A  Salto bueno!
004DE78A  A1 10774F00 MOV EAX,DWORD PTR DS:[4F7710]
004DE78F  8038 00    CMP BYTE PTR DS:[EAX],0
004DE792  74 06      JE SHORT tut_inje.004DE79A   Salto bueno!
004DE794  E8 6FFEFFFF CALL tut_inje.004DE608      Call de los mesajes.
004DE799  C3         RETN
004DE79A  E8 09FEFFFF CALL tut_inje.004DE5A8      Aqui queremos llegar.
004DE79F  C3         RETN
```

Esta ses una interesante zona, si cambiamos el ***Flag Zero*** para que salte cualquiera de los dos saltos entramos en el programa sin más complicaciones, tenemos pues dos flags (variables globales) en los que meter mano, fijáos en las instrucciones del tipo:

CMP BYTE PTR [EAX], 0

En estos dos casos tenemos que ***[EAX]*** es el flag a seguir, sigamos el ejemplo para el primer caso, ponemos un ***BP*** en ***:4DE785*** donde está la comparación, picamos con el botón derecho y elegimos ***Follow in Dump->Memory Address*** para ver a qué dirección representa:

```
Address   Hex dump                                          ASCII
004F88C0  00 01 00 00 FF FF FF FF E4 22 AA 00 14 23 AA 00  .0..     ö"¬.¶#¬.
004F88D0  38 40 AA 00 14 44 AA 00 F0 47 AA 00 CC 4B AA 00  8@¬.¶D¬.-G¬.╠K¬.
004F88E0  A8 4F AA 00 68 53 AA 00 28 57 AA 00 E8 5A AA 00  ¿O¬.hS¬.(W¬.╚Z¬.
004F88F0  A8 5E AA 00 68 62 AA 00 28 66 AA 00 E8 69 AA 00  ¿^¬.hb¬.(f¬.╚i¬.
```

El byte que nos interesa es el que está resaltado debido a que la comparación es del tipo "***BYTE PTR***" además sabemos que debería haber un ***01*** en esta variable para que saltara y le indicara al programa qu no está expirado, además después nos da otra oportunidad, a bote pronto parece que uno es el flag contador de días y otro el que controla el estado del usuario (***Trial***, ***Registered***..) esto se puede pensar ya que si uno está registrado no tiene que ver este cuadro de registro aunque tenga el programa instalado hace un año, probémoslo, antes veamos que dirección esconde la segunda comparación, lo hacemos de la misma manera y vemos:

```
Address   Hex dump                                          ASCII
004F88C1  01 00 00 FF FF FF FF E4 22 AA 00 14 23 AA 00 38  0..     ö"¬.¶#¬.8
004F88D1  40 AA 00 14 44 AA 00 F0 47 AA 00 CC 4B AA 00 A8  @¬.¶D¬.-G¬.╠K¬.¿
004F88E1  4F AA 00 68 53 AA 00 28 57 AA 00 E8 5A AA 00 A8  O¬.hS¬.(W¬.╚Z¬.¿
004F88F1  5E AA 00 68 62 AA 00 28 66 AA 00 E8 69 AA 00 A8  ^¬.hb¬.(f¬.╚i¬.¿
```

En el segundo caso la variable es ***[4F88C1]*** y su valor resaltado en la imagen es ***01***, aquí debería haber un ***00***, tenemos dos variables que controlan la aprición del mensaje Expired y demás, un tanto sospechoso sí que es. De momento reiniciemos, vamos a estas direcciones con ***Ctrl+G*** en la ventana dump y le ponemos los valores que necesitamos y le ponemos un ***BPM On Memory Write*** para que ***Olly*** pare cuando vayan a escribir sobre alguno de ellos, queremos saber quien los modifica y sobretodo donde, veamos donde para:

```
Address   Hex dump   Disassembly                          Comment
0048C634    E8 E3FDFFFF  CALL tut_inje.0048C41C
0048C639    E8 5AECFFFF  CALL tut_inje.0048B298
0048C63E    A2 C0884F00  MOV BYTE PTR DS:[4F88C0],AL      Va a escribir en [4F88C0]
0048C643    8D45 FC      LEA EAX,DWORD PTR SS:[EBP-4]
0048C646    E8 81FCFFFF  CALL tut_inje.0048C2CC
0048C64B    8B55 FC      MOV EDX,DWORD PTR SS:[EBP-4]
0048C64E    B8 BC884F00  MOV EAX,tut_inje.004F88BC
0048C653    E8 A476F7FF  CALL tut_inje.00403CFC
0048C658    E8 A7FDFFFF  CALL tut_inje.0048C404
0048C65D    A2 C1884F00  MOV BYTE PTR DS:[4F88C1],AL      Va a escribir en [4F88C1]
0048C662    33C0         XOR EAX,EAX
```

Precisamente para en una zona en la que va a inicializar los dos flags seguido uno del otro, el primero en pasar por el confesionario se llama ***[4F88C0]*** y será juzgado en el Penal ***:48C63E*** aunque su sentencia se firmó en ***:48C639***, jejeje.. bromas aparte el código canta por sí sólo, dos calls que inicializan nuestras variables, lógicamente el primer ***CALL*** debe retornar ***AL==1*** y retorna justo lo contrario como era de esperarr, pongamos un ***BP*** en sendos calls y reiniciemos, al parar en el primero entraremos, vamos a buscar el mçetodo mças agrasivo de que nos retorne ***AL==1*** no importa si nos hemos de meter con los subcalls para eso están, este valor de retorno será nuestro próximo objetivo y no podemos fallar, adelante entremos, os mostraré como orientaros en calls grandecitos de este tipo:

Prmero de todo ubiquemos el lugar donde ***AL*** tomará su valor final y que será el de retorno de la función.

```
Address   Hex dump   Disassembly                          Comment
0048B3C7  .  C3      RETN                                 Falso retorno, va a :48B3CF
0048B3C8  .^ E9 D382F7FF  JMP tut_inje.004036A0
0048B3CD  .^ EB EB   JMP SHORT tut_inje.0048B3BA
0048B3CF  .  8A45 FF MOV AL,BYTE PTR SS:[EBP-1]           AL toma su retorno..!!
0048B3D2  .  5F      POP EDI
0048B3D3  .  5E      POP ESI
0048B3D4  .  5B      POP EBX
0048B3D5  .  8BE5    MOV ESP,EBP
0048B3D7  .  5D      POP EBP
0048B3D8  .  C3      RETN
```

No hay lugar a dudas que esto ocurre en ***:48B3CF***, vale ahora veamos: lo toma de ***[EBP-1]*** verdad? busquemos donde toma ***[EBP-1]*** su valor, son el botón derecho picamos sobre él y elegimos ***Find References to->Address Constant***, veremos muchos sitios pero pensar que estamos hablando de una "***variable local***" cuyo ámbito está limitado al de la función de pertenencia, es decir que fuera de la presente rutina no tendrá ninguna validez, ok? Veamos el listado es muy grande mostraré lo esencialmente indispensable:

```
0048A054  DD FFFFFFFF
0048B2B7  MOV BYTE PTR SS:[EBP-1],0
0048B33C  MOV BYTE PTR SS:[EBP-1],1
0048B3CF  MOV AL,BYTE PTR SS:[EBP-1]
0048B3DC  DD FFFFFFFF
0048B424  DD FFFFFFFF
0048B450  MOV BYTE PTR SS:[EBP-1],0
0048B507  MOV BYTE PTR SS:[EBP-1],1
0048B50D  MOV BYTE PTR SS:[EBP-1],1
0048B550  MOV AL,BYTE PTR SS:[EBP-1]
0048B558  DD FFFFFFFF
0048B564  DD FFFFFFFF
0048B5AC  DD FFFFFFFF
0048B5BC  DD FFFFFFFF
0048B763  MOV AL,BYTE PTR SS:[EBP-1]
0048B76C  DD FFFFFFFF
```

La rutina comienza en ***:48B298*** y termina en ***:48B3D8*** como se ve en una de las imágenes anteriores, las referencias válidas son las que están resaltadas ya que caen dentro del rango de la rutina, digamos su ámbito, tenemos pues:

```
:0048B2B7 |.  C645 FF 00   MOV BYTE PTR SS:[EBP-1],0      ; Inicialización a ERROR por defecto
:0048B33C |.  C645 FF 01   MOV BYTE PTR SS:[EBP-1],1      ; nuestro objetivo pasar por aqui..!!
```

Está clarísimo que nuestro objetivo como dice el comentario anterior consiste en llegar por todos los medios a la zona de la asignación de nuesrta variable a ***1***, esta es sin duda la activación del flag de éxito que le indica al programa que las cosas van bien, esto es justamente lo que queremos y para esto estamos aquí, ahora no lo va a poner fácil, ya de entrada veo muchos ***CALL*** que se entrometerán en nuestro camino pero los iremos despachando de igual manera, recordad de entrar a matar, no nos vale cambiar saltos superficiales, ***MODIFIQUEMOS LOS RETORNOS***..!! este es el único modo de parchear correctamente una aplicación y que no te dé problemas a

posteriori cuando intentes utilizar una función de las reservadas para usuarios registrados, este es un detalle importante a tener muy en cuenta y por eso lo recalco, tras una ligera pasada de reconocimiento concluimos que el único call que molesta es este:

```
Address   Hex dump    Disassembly                  Comment
0048B333  .  E8 38050000  CALL tut_inje.0048B870    Debe retornar AL==1..!!
0048B338  .  84C0         TEST AL,AL
0048B33A  .v 74 06        JE SHORT tut_inje.0048B342
0048B33C  .  C645 FF 01   MOV BYTE PTR SS:[EBP-1],1   Flag de exito..!!
0048B340  .v EB 3A        JMP SHORT tut_inje.0048B37C
0048B342  >  8D4D F0      LEA ECX,DWORD PTR SS:[EBP-10]
```

Tracear vosotros mismos y veréis como llegamos con cierta comodidad a esta zona, mejor para nosotros, nos vamos a sumergir en un nivel inferior donde seguiremos el trabajo, nos anotamos que ahora el retorno será ***AL==1***, venga vamos, a diferencia del call anterior este no es muy grande y sólo muestra un salto condicional a mitad más o menos del mismo, si miramos los registros vemos que está manejando cadenas más que sospechosas, llegamos al salto y no se produce, como dije antes busquemos el lugar donde ***AL*** toma su valor de retono:

```
Address   Hex dump    Disassembly                  Comment
0048B8EC  .  8BC3         MOV EAX,EBX                AL tomal valor de retorno..!!
0048B8EE  .  5B           POP EBX
0048B8EF  .  8BE5         MOV ESP,EBP
0048B8F1  .  5D           POP EBP
0048B8F2  .  C3           RETN
```

Aquí es ***EAX*** el que toma el valor y como ***AL*** es la parte baja de este registro se está modificando implícitamente, luego este es el lugar buscado, en este caso el que le pasa el datro es ***EBX*** y es el registro a seguir en el call a ver de donde toma su valor:

```
Address   Hex dump    Disassembly                  Comment
0048B8C3  .  E8 34F4FFFF  CALL tut_inje.0048ACFC
0048B8C8  .  8BD8         MOV EBX,EAX                Aqui EBX toma valor de EAX..!!
0048B8CA  >  33C0         XOR EAX,EAX
0048B8CC  .  5A           POP EDX
```

Pues es aquí y lo toma de ***EAX***, o sea que digamos que le reserva el valor hasta la salida porque ya no se vuelve a modificar, entonces ya sabemos que el retorno del ***CALL*** depende del valor de ***EAX*** en ***:48B8C8*** y éste lo toma del call anterior, nos debemos pues meter en este otro call y forzar su retorno a ***EAX==1***, notad como nos vamos introduciendo en sucesivoa anidamientos, esto es muy bueno ya lo veréis, si miramos los registros en el preciso momento que vamos a entrar en el call vemos:

```
Registers (FPU)
EAX 00AA3044 ASCII "22317FB0138A79"
ECX 00000000
EDX 00AA5ED8 ASCII "123213213231321321231231321321321"
EBX 00000000
ESP 0012FEF4
EBP 0012FF10
ESI 0000007D
EDI 004DE7D0 tut_inje.004DE7D0

EIP 0048B8C3 tut_inje.0048B8C3
```

justamente el de abajo es uno que puse en el cuadro de registro por probar, parece ser que lo guardó en algún lugar (suguro que en el registro del sistema) y ahora lo ha recuperado, y mirar el de arriba parece un serial, sabemos que no lo es por su formato ya que no tiene el pedido en el cuadro de registro, entremos dentro del call y ya sabemos que el objetivo se llama ***EAX==1***…

```
Address   Hex dump    Disassembly                  Comment
0048AD60  .  E8 C391F7FF  CALL tut_inje.00403F28
0048AD65  .  83F8 06      CMP EAX,6
0048AD68  .v 7D 0A        JGE SHORT tut_inje.0048AD74
0048AD6A  .  E8 098AF7FF  CALL tut_inje.00403778
0048AD6F  .v E9 F7030000  JMP tut_inje.0048B16B
0048AD74  >  8D55 C8      LEA EDX,DWORD PTR SS:[EBP-38]
0048AD77  .  8B45 F8      MOV EAX,DWORD PTR SS:[EBP-8]
```

ojo que esta es la primera restricción, está sin duda validando nuestros datos de registro..!! yo puse ***SyXe'05*** como nombre que si contáis son 7 letras, mirando en los registros vemos ***EAX==7***, jeje.. los está validando, sigamos que esto se está poniendo muy interesante, viene una enorme batería de calls, los pasamos rápidamente pues no tienen comparación a la salida, imagino que se estará preparando el serial válido o una parte no sé, además parece que se están desencriptando las cadenas, al final llegamos a la zona de comparaciones y atención:

```
Address   Hex dump    Disassembly                  Comment
0048B139    E8 FA8EF7FF   CALL tut_inje.00404038
0048B13E  v 75 11         JNZ SHORT tut_inje.0048B151
0048B140    8B45 D4       MOV EAX,DWORD PTR SS:[EBP-2C]
0048B143    8B55 D0       MOV EDX,DWORD PTR SS:[EBP-30]
0048B146    E8 ED8EF7FF   CALL tut_inje.00404038
0048B14B  v 75 04         JNZ SHORT tut_inje.0048B151
0048B14D    C645 F7 01    MOV BYTE PTR SS:[EBP-9],1   Flag de exito activado..!!
0048B151    33C0          XOR EAX,EAX
```

Esta es sin duda la zona más importante de todo el código que hemos visto hasta ahora, mirad el retorno de donde lo toma ***AL***:

```
Address   Hex dump    Disassembly                  Comment
0048B1AA    8A45 F7       MOV AL,BYTE PTR SS:[EBP-9]  AL coge su valor de retorno..!!
0048B1AD    8BE5          MOV ESP,EBP
0048B1AF    5D            POP EBP
0048B1B0    C3            RETN
```

Hombre pero si lo toma de ***[EBP-9]***, otra variable local del procedimiento, jajaja… retornemos a la anterior imagen y mirad que hay justo debajo de los dos calls, la activación del flag a 1 que es nuestro retorno válido de la fucnión..!! teniendo en cuenta el nivel de anidamiento en que nos encontramos estoy seguro que salir de aquí con ***AL==1*** pondrá en grandes apriteos la seguridad del programa, repito que ese es sin duda uno de los mejores puntos para atacarle a ***Registry Medic***, lo haremos además por la puerta grande: Modificando los retornos de los dos ***CALLS*** previos a la activación del flag, en el primero de ellos sólo hay que mirar los registros para darse cuenta de la suma

importancia que tiene este fragmento de código en el posterior desarrollo del progarma, estoy convencido que de él depende en gran medida nuestro éxito o fracaso, mirad y juzgar por vosotros mismos:

```
Registers (FPU)
EAX 00AA38CC ASCII "266ab7aa8c1"
ECX 00000000
EDX 00AA7E98 ASCII "12321321323"
EBX 00000000
ESP 0012FE28
EBP 0012FEEC
ESI 0000007D
EDI 004DE7D0 tut_inje.004DE7D0
EIP 0048B139 tut_inje.0048B139
```

No os suena ese ***12321321323***, es un fragmento del serial que puse y que antes vimos completo, lo que ahora sí coinciden en número de dígitos ambos seriales, antes ejecutó muchos calls que seguramente desencriptaron la cadena mayor que traía recordais y llegó a la conaclusión de que el otro serial que trae ***EAX*** debe ser equivalente al trocito de nuestro serial, si entramos en el call vemos que claramente compara ambos así que este es un fragmento de serial válido, podríamos serguir esta línea y formar el válido completamente, el retorno el lógicamente ***EAX==0*** y quiere saltar en ***:48B13E***, no le dejaremos, jejeje…

Al no saltar porque le cambiamos el flag zero se cargaron en los registros el resto de serial que necesita validar y que es:

```
Registers (MMX)
EAX 00AA7EB0 ASCII "321"
ECX 61363632
EDX 00AA5F6C ASCII "762"
EBX 00000000
ESP 0012FE28
EBP 0012FEEC
ESI 0000007D
EDI 004DE7D0 tut_inje.004DE7D0
EIP 0048B146 tut_inje.0048B146
```

Si hubieran coincidido los seriales anteriores ahora iría a verificar estos ***3 dígitos***, ahora lo va a hacer pero porque cambié el ***Flag Zero*** sino no hubiera llegado a esta parte, sabemos pues que de la cadena mayor la desencripta, hace sus cuentas y compara lo que hemos puesto como serial con lo que deberíamos haber puesto, esto se pone interesante, muuuuuuuuuy interesante ;-)

Si os digo la verdad yo también estoy algo sorprendido de ver las cosas así de claras, os he de confesar que al final conseguí el crack del programa pero me costó mucho, sobretodo para que hiciera la función de repara el registro y todo parcheando, estos dos saltos los nopeé sin darme cuenta que tenía la respuesta en mi propia cara, estudiemos el primer salto, tenemos los registros:

EAX==266ab7aa8c1
EDX==12321321323

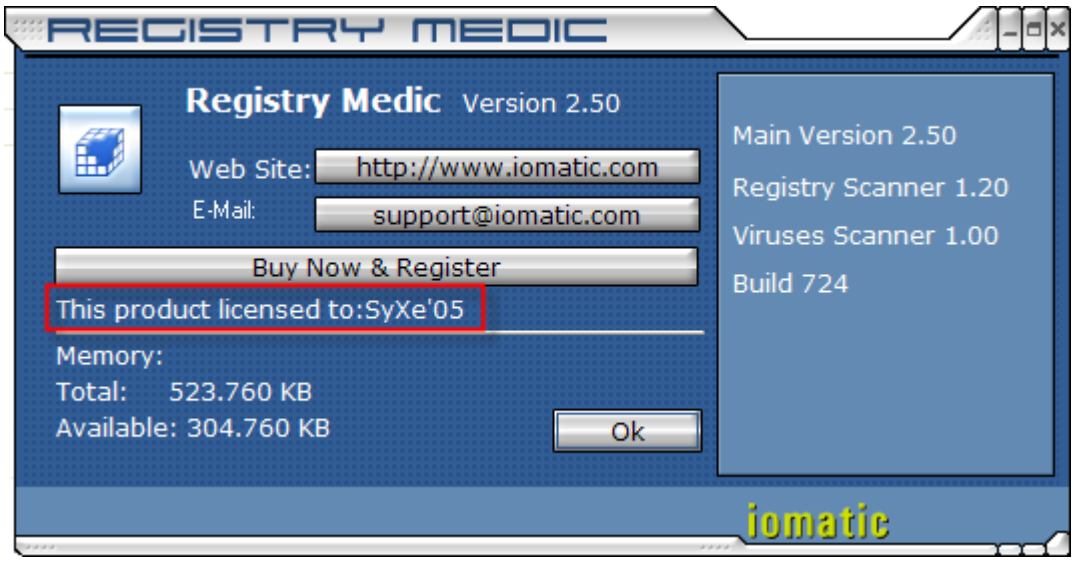
El contenido de ***EDX*** es parte del serial que guardó en el registro cuando falseé la rutina para que lo aceptara, en caso contrario creo que no lo guarda porque lo he intentado cambiar poniendole otro pero ni caso se resiste, he tenido que hacerlo a nuestro modo ;-) al final he conseguido que lo guarde, esto porque el serial anterior es del tipo ***123231123***… y no se pueden distinguir las posiciones en las que mira, por eso hice que cogiera este otro mejor:

SyXe'05 ; Como nombre
01234-56789-ABCDE-FCCBB-44223-55667-762 ; como serial

Cuando para en el primer salto veo que las condiciones no varían, pues sigue comparando con "***266ab7aa8c1***" y lo que comparaba eran los ***11*** primeros caracteres de mi serial pues ahora sí se podían distinguir entre ellos, o sea que el serial debe comenzar con esos caracteres, vale perfecto, luego las tres que mira después son los tres çultimos, perfecto, es de mencionar que van variando si cambiamos algún carácter del serial, o sea que estos tres últimos dígitos se calculan a partir de los anteriores, vale me anoté lo que me pedía y se lo puse:

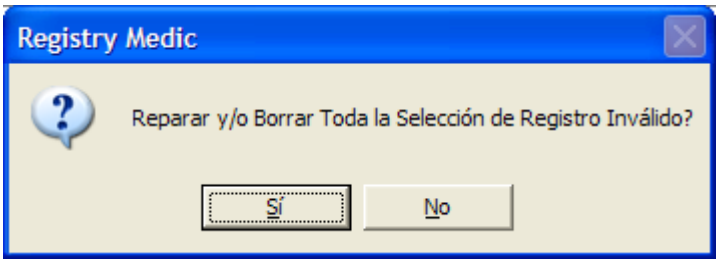
Nombre: SyXe'05
Serial: 266AB-7AA8C-10123-45678-9ABCD-EFFEB-902

Del nombre nos pide que tenga almenos 6 dígitos si recordáis y el serial lo valida en esta parte, pongo estos datos de registro y me dice "***invalid key***" lo fuerzo a que lo acepte y lo guarde en el registro y cuando reinicio ya que lo hace por cuenta propia pues se cierra, veo que con esta combinaciín ***Name/Serial*** se activan los flags ***[4F88C0]*** y ***[4F88C1]***, no muestra el mensaje de ***Expired*** y carga el programa, voy al ***About*** y veo mi nombre allí como ***Usuario Registrado***:



Más claro el agua, primero da por válido el serial y luego se ejecuta con todas las opciones activadas parece ser, en el About nuestro nombre, la opción de registro queda deshabilitada, muestra un color apagado como con gota de agua, o sea que todo indica que estamos registrados.!! jajaja… lo conseguimos

Me aseguraré, hago un análisis completo del registro cuando termina tiene todo un aspecto espantoso pues encontró mas de ***100*** entradas mal, jodidas vaya, bueno me fijo en una mala poruqe te da la opción de abrir el registro, me pongo la clave a la vesta, le digo que la elimine y me sale este cartelito primero:



Lógicamente le digo que sí y como si parpadeara un instante, parace que no ha pasado nada pues ya terminó y la clave sigue ahí ;-(pero … actualizo con ***F5*** y desaparece… ***REALIZA LA FUNCIÓN***..!!

He de decir que he conseguido el serial válido a la par que escribía el tut pues para hacer las capturas de pantalla he repetido el proceso que he hecho al crackearlo y ya os digo que ha sido algo casual, de hecho el tut iba en dirección de un parche, en total había que parchear en ***17*** sitios, y lo peor fue conseguir que hiciera la función que acaba de hacer, en fin que se le dio a todo la vuelta que se le va a hacer, mejor un serial válido que no un parche de toda la vida, he dedecir además que el desempakado nos lo podríamos haber evitado, pero bueno práctica que vamos cogiendo, siempre es bueno saber descomprimir los programas a mano.

Y al fin terminamos, al sacar el serial válido el tema se acorta muchísimo, para explicar todos los pasos del parche me hubiera llevado mucho más tiempo, mejor para nosotros.

Si alguien tiene alguna duda acerca del desempaquetado o del método utilizado paraobtener el sreial me puede escribir, siempre digo que lo importante no es cambiar saltos superficiales, de poco vale creedme, tenemos casi la obligación de meternos lo más profundo posible en el código, hoy mismo ha sido un ejemplo de lo que os digo, sólo cambiando saltos nunca hubiéramos dado con el serial válido. El que necesite lo que sea me puede escribir a mi dirección de correo electrónico:

syxe00@yahoo.es
syxe00@hotmail.com

Como siempre deciros que contestaré todos y cada uno de los e-mails recibidos, hasta la próxima que sea pronto, chao… Y desde aquí mando un fuerte saludo a todo ***[CracksLatinos]…***